Principles of Computer Science II

Capture the flag: recursion, classes and objects

Marco Zecchini

Sapienza University of Rome

Tutorial 4

Today's plan

- One game, in **teams of three**: a *capture the flag*.
- Topics: **recursion** (Lecture 2, Tutorial 3) and **classes and objects** (Lecture 4). Nothing new: you combine what you have.
- Six exercises in a **chain**: the output of each one is the input of the next. At the end of the chain there is a **flag**.
- The first teams to capture the flag win.

Why a game? Because the hard part of programming is not the syntax: it is agreeing on what a function receives, what it returns, and who does what. Today you practise exactly that.

What is a capture the flag?

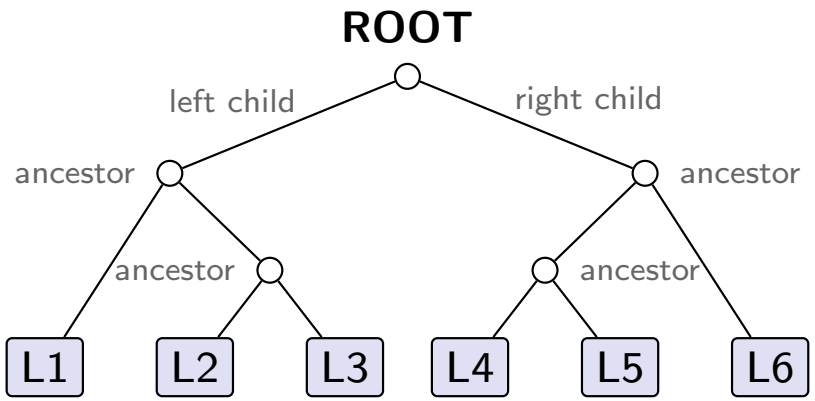
- A format born in computer security competitions: a set of challenges, each hiding a secret string, the **flag**.
- You solve the challenge, you obtain the flag, you submit it; a server checks it automatically. No discussion about whether an answer is "almost right".
- Our flags look like `PCS{84746107dc}`: the string is computed from your team name and your answer, so every team has a different one.
- Our variant: not independent challenges, but a **chain** of six, and the information needed to solve it is **split among the three members** of the team.

The story

- Six samples, `L1` . . . `L6`, from six organisms.
- A small **phylogenetic tree** relates them: leaves are samples, internal nodes are unknown ancestors.
- Goal: the DNA of the **common ancestor** at the root, 12 bases. That sequence is your **answer**; the notebook turns it into the flag.
- Real data is never clean: one read is **contaminated**, one was read on the **reverse strand**, one sample is **not in the tree**, and the reads come as **RNA**.

```
ROOT
|-- (ancestor)
|   |-- L1
|   `-- (ancestor)
|       |-- L2
|       `-- L3
`-- (ancestor)
    |-- (ancestor)
    |   |-- L4
    |   `-- L5
    `-- L6
```

The tree




```
ROOT
|-- (ancestor)
|    |-- L1
|    `-- (ancestor)
|         |-- L2
|         `-- L3
`-- (ancestor)
     |-- (ancestor)
     |    |-- L4
     |    `-- L5
     `-- L6
```

- The tree is the **same for every team**; what changes is the data on its leaves. The Archivist's email has it as **text**, as on the right, and says which sample is marked `*`.
- Reading the text: each `|--` / `` `-- `` line is a **child** of the nearest line above with less indentation; the **first** is the **left** child. `(ancestor)`: internal node, unnamed.
- In Python, one `TreeNode(...)` per line: `TreeNode("ROOT", TreeNode("anc", TreeNode("L1"), ...), ...)`.

The three roles

Each member of the team receives **one email** with one piece of the puzzle.

Role	What you receive	Needed in
A Archivist	the **tree** as text, with one sample **marked**	Exercises 3, 4
B Sequencer	the **RNA reads** of the samples and a quality rule	Exercises 1, 3, 4
C Theorist	a **reference sequence** and the **rule** that computes an ancestor	Exercises 2, 6

The rule of the game

Your email is yours alone: do **not forward** it, do not paste it anywhere. You **explain** what you have to your teammates, by voice. Nobody can finish alone.

How the game works

1. Form a **team of three**; choose a **team name**.
2. **One** of you fills in the form: team name and three emails, one per role. Decide the roles together.
3. Within a minute, each of you receives **your email**. Read it; do not show it.
4. Open the **Colab notebook** (one per team), save a copy, write the team name *exactly* as in the email in the `TEAM` cell, and follow the **chain** of six exercises.
5. `submit_flag(answer)` asks the server: **CORRECT** or **WRONG**. Every attempt is recorded, with the time.

`https://forms.gle/7uRP5yp3DDX9WQpW6`

The chain

#	Exercise	Topic	Uses
1	`reverse_transcribe(rna)`: RNA → DNA	recursion, strings	B
2	`count_base(seq, base)` → priority of the bases	recursion	C
3	class `DNASequence`: GC content, quality, reverse complement	classes	B, A
4	class `TreeNode` from the drawing, `leaf_labels()`	classes, trees	A
5	method `attach(reads)`: a read on every leaf	objects	–
6	method `ancestor(priority)` → sequence at `ROOT`	recursion, trees	C

- Every exercise: a **test cell** on a tiny public example (run it first), then a cell **"Now your data"**, where the role holder dictates.
- **Checkpoints**: a wrong Exercise 1 gives absurd GC contents in Exercise 3; a wrong tree in Exercise 4 does not match the samples of B.

How to work

1. **Roles decide who holds which information, not who works.** The role holder dictates and explains; the whole team writes and checks the code.
2. **Talk first, type later.** For every exercise: base case, recursive case, **pseudocode on paper**, then Python. Exactly as in the exam.
3. **One notebook per team.** One person types, the others read and check; **swap** at every exercise.
4. **Agree on the interface** before coding: what does `ancestor` receive? What does it return when a sample was discarded? The Theorist knows the rule, but the Archivist must check that it is well defined on the tree.
5. **Stuck?** Read the error message, then ask. **Wrong flag?** Walk the chain backwards: rule, tree, reverse-strand sample, contaminated sample, priority, reverse transcription.

What you need from the lectures

Recursion (Lecture 2)

- a **base case**, solved without further calls;
- a **recursive case**, on a smaller input: `s[1:]`, `s[:-1]`, a child of the node;
- on a tree: ask the two children, **combine** the answers.

Classes (Lecture 4)

- `class`: a template; an object is one instance;
- `__init__`: the constructor; `self`: the object a method is called on;
- an attribute may hold **another object**, or `None`: `node.left`.

Example: `tiny = TreeNode("ROOT", TreeNode("L1"), TreeNode("anc", TreeNode("L2"), TreeNode("L3")))`, and `tiny.leaf_labels()` is `["L1", "L2", "L3"]`.

Start

Colab notebook of the challenge

`https://drive.google.com/file/d/133pnjF-6lrupR8bT4pHbxK2f8YWj27PK/view?usp=sharing`

1. Teams of three, a team name, one form per team: `https://forms.gle/7uRP5yp3DDX9WQpW6`
2. Check your inbox (and the spam folder). Do not share your email.
3. `File > Save a copy in Drive`; fill in `TEAM`; run the helper cell.
4. Exercise 1. Pseudocode first.

Winner: the first team with a **CORRECT** flag. Then we look together at how the chain works, and at the complexity of `ancestor`.